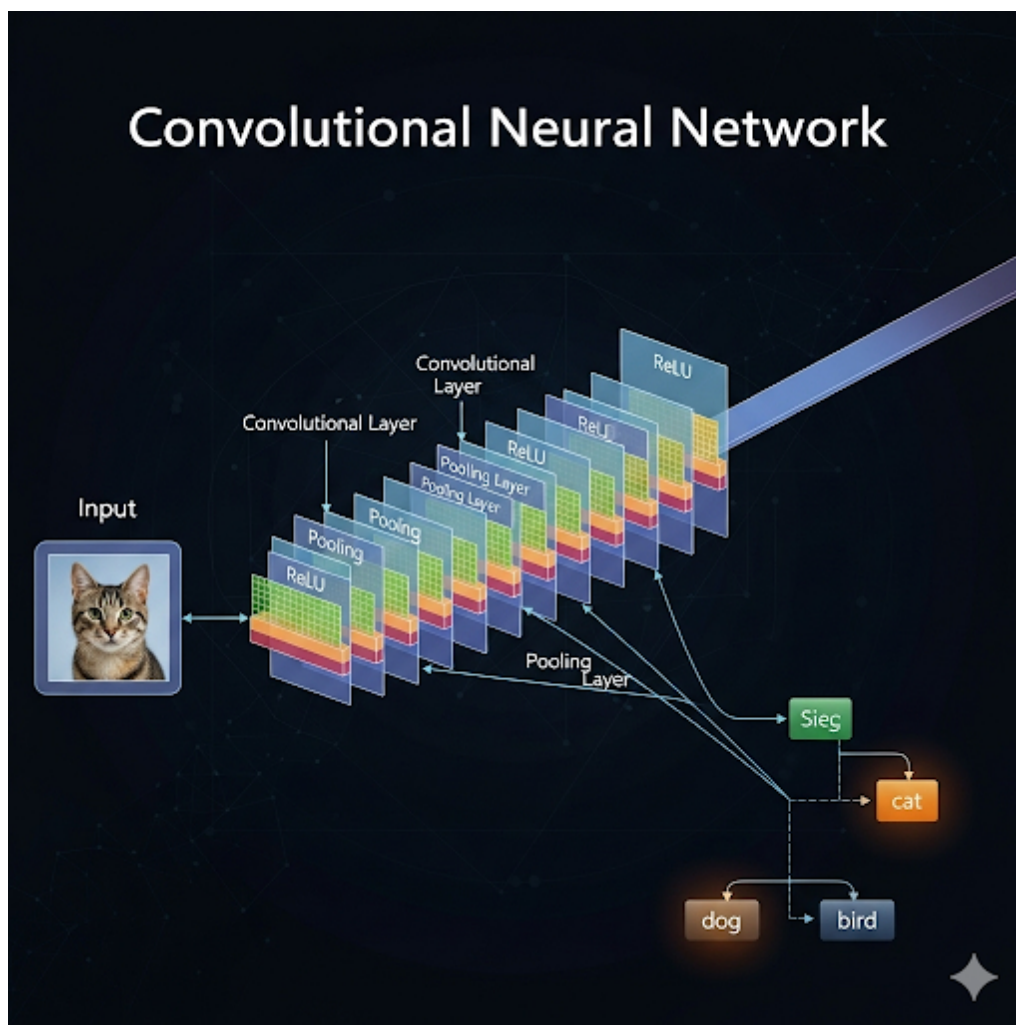


AI : Understanding Convolutional Neural Networks

 medium.com/@angadi.saa/ai-understanding-convolutional-neural-networks-912a6a6d6120

Shankar Angadi

September 11, 2025



The Architecture That Revolutionized Computer Vision

How CNNs solved the pixel adjacency problem and transformed AI's ability to see

When we humans look at an image, we intuitively understand that nearby pixels are related to each other. A cat's whisker consists of adjacent pixels forming a line, and a human face is composed of features where spatial relationships matter enormously. But for early neural networks, this seemed like an impossible problem to solve elegantly.

The Fundamental Problem with Traditional Neural Networks

Imagine trying to recognize a cat in a photo using a traditional fully connected neural network. If you treated each pixel as an independent input, something bizarre would happen: the network would produce the exact same result whether you showed it a normal cat photo or the same photo with all pixels randomly scrambled like a jigsaw puzzle thrown in the air.

This reveals the core issue: **traditional neural networks completely ignore spatial relationships**.

But the problems run even deeper. Consider a typical megapixel RGB image with roughly one million pixels. In a fully connected network where each pixel connects to each unit in the first hidden layer, you'd need about **9 trillion weights** just for that first connection. The computational and data requirements would be astronomical — you'd need correspondingly vast training datasets and enormous computational budgets.

Clearly, we needed a better approach.

Enter Convolutional Neural Networks: A Stroke of Genius

Convolutional Neural Networks (CNNs) solve this elegantly by incorporating two key insights about how vision actually works:

1. Locality Matters (Local Connectivity)

Instead of connecting every pixel to every neuron, CNNs connect each hidden unit to only a small, local region of the image. This immediately:

- : Nearby pixels that should influence each other actually do
- : Instead of n^2 weights, we now have only ln weights (where l is much smaller than n)

Think of it like looking through a small window that slides across the image, focusing on local patterns rather than trying to process everything at once.

2. Patterns Repeat Everywhere (Spatial Invariance)

Here's the crucial insight: **anything detectable in one part of an image should be detectable anywhere else**. An eye looks like an eye whether it appears in the top-left corner or bottom-right corner of a photo. A blade of grass has the same characteristics regardless of where it grows in the frame.

CNNs exploit this by using the same set of weights (called a **kernel**) across different regions of the image. This means:

- The same feature detector works everywhere

- We need even fewer parameters (down to just dl weights, independent of image size!)
- The network learns universal feature detectors

How Convolution Actually Works

Let's break down the convolution operation with a simple example. Imagine a 1D image (like a single row of pixels) with values [5, 6, 6, 2, 5, 6, 5] and a kernel [+1, -1, +1].

The convolution process slides this kernel across the image:

- At each position, it computes the dot product between the kernel and the local pixel values
- With a stride of 2 (moving 2 pixels at a time), we get outputs that detect specific patterns
- A kernel like [+1, -1, +1] might detect darker points surrounded by lighter areas

This seemingly simple operation can be expressed as efficient matrix multiplication, making it perfect for modern GPU acceleration.

The Power of Pooling: Seeing the Forest and the Trees

CNNs often include **pooling layers** that summarize information from adjacent units:

Average Pooling computes the mean of local regions, effectively downsampling the image while preserving general structure. This enables **multiscale recognition** — the same classifier that recognizes a small object can now recognize that object even when it appears larger in the pooled version.

Max Pooling takes the maximum value from each region, acting like a logical OR operation. It answers the question: “Does this feature exist somewhere in this region?” This is incredibly useful for detecting features regardless of their exact position.

The Magic of Receptive Fields

One of the most elegant aspects of CNNs is how **receptive fields** grow with depth. In the first layer, each neuron “sees” only a tiny local region (the kernel size). But as you go deeper:

- Second layer neurons can see larger regions
- Third layer neurons see even larger areas
- Deep layers can capture global patterns and relationships

With stride=1, receptive field size grows linearly with depth. With larger strides, it grows exponentially. This creates a natural hierarchy from local features to global understanding.

Tensors: Keeping Track of Structure

CNNs work with **tensors** — multidimensional arrays that preserve the spatial structure of data. For example:

- Input: A $256 \times 256 \times 3 \times 64$ tensor (width $\times$ height $\times$ color channels $\times$ batch size)
- After convolution with 96 kernels: A $128 \times 128 \times 96 \times 64$ tensor
- The result: 96 showing how different features appear across the image

Each channel in these feature maps captures different aspects — some might detect edges, others might detect textures or more complex patterns. The beauty is that the network learns what features are most useful for the task at hand.

Why CNNs Changed Everything

The introduction of CNNs represented a paradigm shift because they **inject prior knowledge about vision** directly into the network architecture:

1. : Nearby pixels are related
2. : Features should be detectable anywhere
3. : Simple features combine into complex ones

This isn't just theoretical — it has practical implications. CNNs require:

- Far fewer parameters than fully connected networks
- Much less training data
- Significantly less computational power
- And they generalize much better to new images

The Bigger Picture

CNNs don't just work for images. The same principles apply to any data where adjacency matters:

- : Temporal patterns in sound waves
- : Sequential relationships in data
- : Relationships between nearby words

The core insight — that structure in data should be reflected in network architecture — has influenced virtually every major advance in deep learning since.

Convolutional Neural Networks solved one of the fundamental challenges in computer vision by recognizing that the architecture of our models should reflect the structure of our data. By respecting locality and spatial invariance, CNNs don't just process images — they *understand* them in a way that mirrors how biological vision systems work.

The next time you use photo recognition on your phone, remember that it's likely powered by these same principles: local connectivity, shared weights, and the elegant mathematics of convolution that transforms raw pixels into meaningful understanding.



Enjoyed the Article? Let's Stay Connected!

If you found this helpful, here's how you can support and stay in the loop:



Clap — it lets the Medium algorithm know this article was worth reading.



Drop a comment — I'd love to hear your take, questions, or even a quick "thanks"!



Highlight your favorite part — it helps other readers and sparks discussions.



Follow me and Let's Code Future for more hands-on tutorials, AEM, AI deep dives, and practical dev content.



Share it with your network — help someone else discover something valuable today!